

CS633 Assignment 1 Report (Group 74)

Rishab K S (Leader) – 251110061
Kanchi Kandadai Bharath – 251110042
Hemant Jagtap – 251110037
Abhishek Dixit – 251110003
Ashwin Kansagara – 251110019

14-02-2026

1 Code Description

In this assignment, we wrote an MPI program using only point-to-point communication (`MPI_Send` and `MPI_Recv`) as specified in the problem statement.

Each process starts with a buffer of M double values. During every iteration, a process may act as a sender and exchange data with two other processes located at distances $D1$ and $D2$ to the right, where $D1 < D2$.

A rank is considered a valid sender if the destination process exists, i.e.,

$$rank + D1 \leq P - 1$$

Similarly, communication with the $D2$ receiver is performed only when

$$rank + D2 \leq P - 1$$

Computation at Receiver Processes

After receiving the buffer, the receiver performs a computation and sends the updated array back:

- Receiver at distance $D1$ computes element-wise square:

$$x[i] = x[i]^2$$

- Receiver at distance $D2$ computes element-wise logarithm:

$$x[i] = \log(x[i])$$

Buffer Update at Sender

Once the sender receives the computed arrays back, it updates its local buffers:

- For $D1$, the returned values are reduced using modulo:

$$buffer_D1[i] = (unsigned\ long\ long)\ x[i] \bmod 100000$$

- For $D2$, the returned values are scaled:

$$buffer_D2[i] = x[i] \times 100000$$

This entire exchange is repeated for T iterations.

Final Maximum and Output

In the final iteration, each valid sender computes the maximum value from the returned buffers (before applying the update operations). These local maxima are sent to Rank 0.

Finally, Rank 0 prints:

$$< maximumD1 > \quad < maximumD2 > \quad < execution\ time >$$

The runtime is measured using `MPI_Wtime()`.

2 Compilation and Execution

The code was compiled on the PARAM Rudra cluster using:

```
mpicc src.c -o src -lm
```

A typical execution command is:

```
mpirun -np P ./src M D1 D2 T seed
```

For example:

```
mpirun -np 16 ./src 262144 2 4 10 1000
```

The fixed parameters used in all experiments were:

- $D1 = 2$
- $D2 = 4$
- $T = 10$
- $seed = 1000$

We tested the required six configurations:

- Process counts: $P = 8, 16, 32$
- Buffer sizes: $M = 262144$ and $M = 1048576$

Each configuration was executed five times. All runs were performed using 16 processes per node, as required.

3 Results

The execution times from all runs were stored in `timing.csv`. A single boxplot was generated to compare the timing distributions across different process counts and buffer sizes.

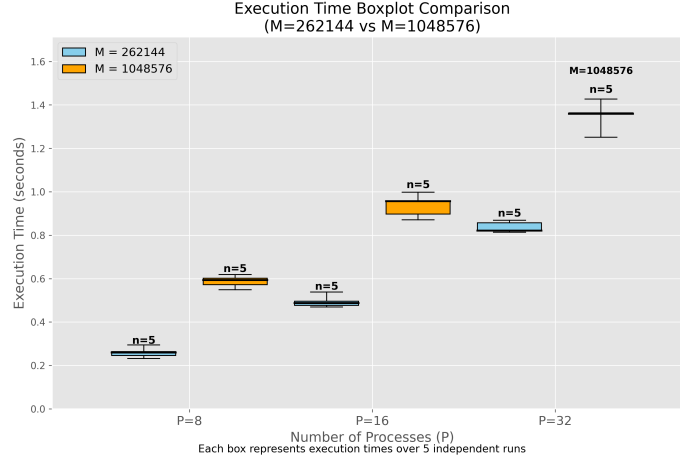


Figure 1: Execution time distributions for $M = 262144$ and $M = 1048576$ across $P = 8, 16, 32$

Figure 1 shows that larger buffer sizes consistently require more time, and the spread across the five runs is relatively small.

Observations

- The execution time increases noticeably when the buffer size grows from $M = 262144$ to $M = 1048576$, since both computation and communication volume increase.
- For both buffer sizes, the runtime increases when the number of processes increases from $P = 8$ to $P = 32$. This suggests that communication overhead dominates the overall execution time for this workload.

- The effect of communication becomes more significant at higher process counts, which limits scalability even though more processes are available.
- The boxplots show relatively small variation across the 5 repeated runs, indicating stable performance on the cluster.

4 Conclusion

In this assignment, we successfully implemented the required MPI communication pattern using only point-to-point primitives (`MPI_Send` and `MPI_Recv`).

From the experimental results, we observe that increasing the buffer size leads to higher execution time due to increased computation and data exchange.

Additionally, increasing the number of processes from 8 to 32 does not reduce runtime. Instead, the execution time increases, showing that the program performance is limited by communication overhead in this communication-intensive pattern.

This assignment highlights the importance of communication costs when scaling MPI applications across more processes.